

MiniOrangeOS操作系统设计与实现

操作系统课程设计

指导教师：王冬青

李晨恺
2026.8.23

一、项目简介与目标

MiniOrangeOS 是一个运行在 x86-64 QEMU 环境中的单核操作系统。系统从 MBR 和二级 Loader 启动，进入 64 位高半区内核，最后运行 Ring 3 Shell 和用户程序。

启动

MBR / Loader
E820 / 长模式
高半区

内存隔离

页帧 / 四级页表
独立 CR3
VMA / COW

进程并发

fork / exec
wait / 线程
调度 / IPC
mutex

存储与终端

ATA LBA48
MyFS / TTY
framebuffer

用户环境

crt0 / libc
管道 / 重定向
20+ 程序

36 个系统调用

shell控制台

完整测试

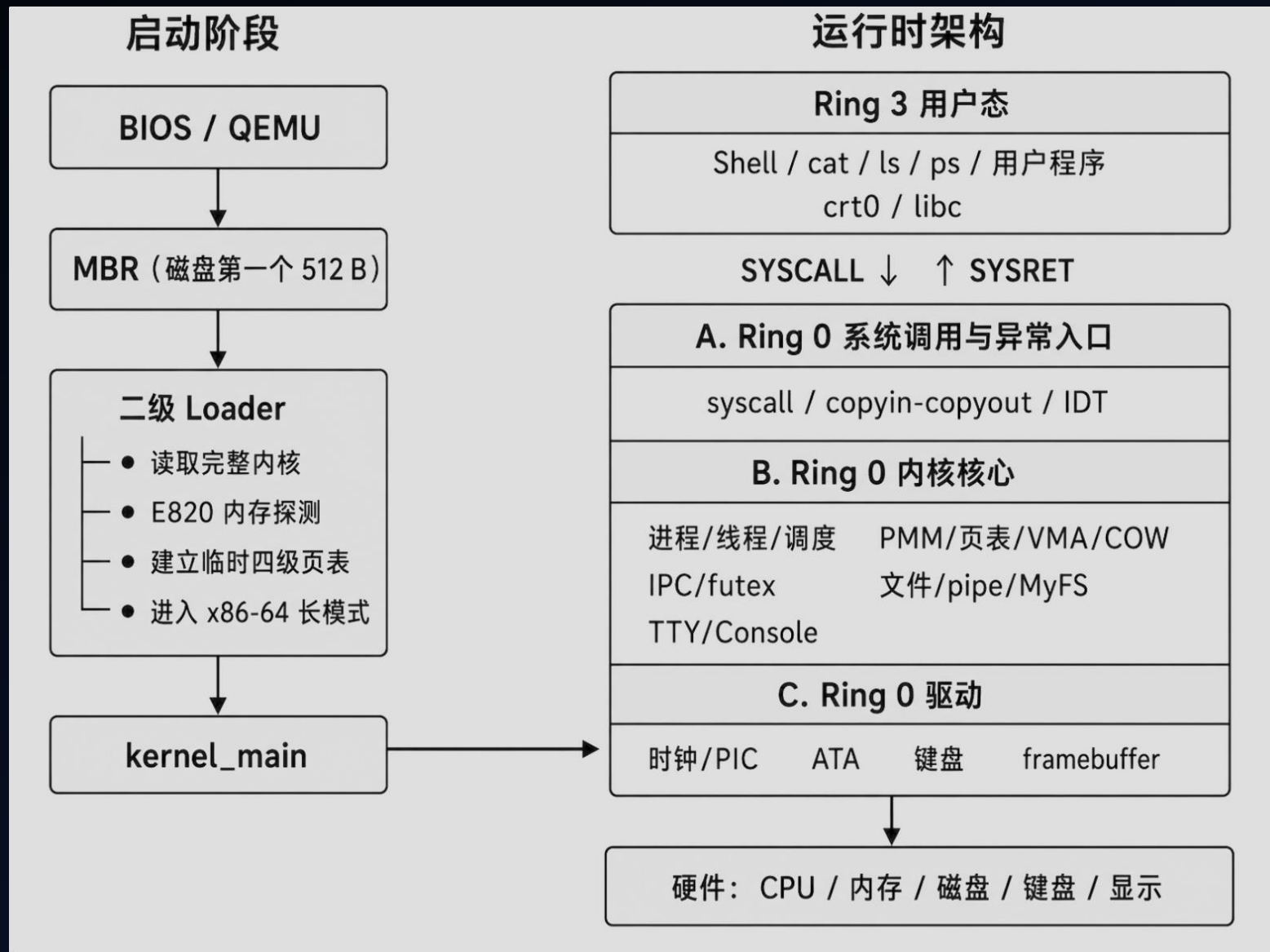
二、启动阶段建立环境，运行阶段按特权级分层

Boot: 加载内核并进入 64 位模式

Ring 0: 管理内存、进程、文件和硬件

Ring 3: 运行 Shell 和普通用户程序

SYSCALL / SYSRET: 用户程序进入内核并返回



三、Loader 动态加载内核并进入 64 位高半区

启动流程：

BIOS 加载 MBR 到 0x7C00

↓

MBR 读取 Loader 和内核前 42 个扇区

↓

Loader 读取剩余内核并执行 E820 探测

↓

打开 A20, 建立 GDT 和临时四级页表

↓

CR4.PAE → EFER.LME → CR0.PG

↓

进入 64 位高半区 kernel_main

磁盘布局：

LBA 0 MBR

LBA 2~9 Loader

LBA 10~ Kernel

LBA 1000~ MyFS

$\text{kernel_sectors} = (\text{kernel_bytes} + 511) / 512$

磁盘边界：Kernel 不能覆盖 MyFS

物理边界：Kernel + .bss 不能覆盖 0x70000 启动页表

超过 255 个扇区时分块读取

300 扇区测试内核成功进入 Ring 3

四、四级页表与 COW

内存结构

物理内存: 4 KB 页帧

页表层级: PML4 → PDPT → PD → PT → Page

每个用户进程: 独立 CR3、用户页表和 VMA

所有用户进程: 共享受保护的内核高半区映射

PTE 权限: Present / Writable / User / COW

VMA 与页错误

匿名 mmap: 分配清零页并建立 VMA

munmap: 解除映射并减少页引用

用户栈: 初始映射 1 页, 最多向下增长 8 MB

Page Fault处理:

1. COW 写错误 → 分离共享页
2. 栈 VMA 缺页 → 分配新栈页
3. 其他用户错误 → 结束当前进程
4. 内核错误 → panic

COW

fork 前: Parent —RW—> Page P = 0x1111

fork 后: Parent —RO+COW—┐
 └—> Page P
 Child —RO+COW—┐

Child 写入后

Parent → Page P = 0x1111, Child → Page Q = 0x2222

COW = PTE 只读标记 + 物理页引用计数 + Page Fault 分离

五、进程与线程

Process	Thread
PID、父子关系	TID、调度状态
CR3、VMA	寄存器上下文
cwd、文件描述符表	内核栈、用户栈
线程链、退出码	TLS、join/detach 状态

抢占式调度：时钟中断提供抢占点

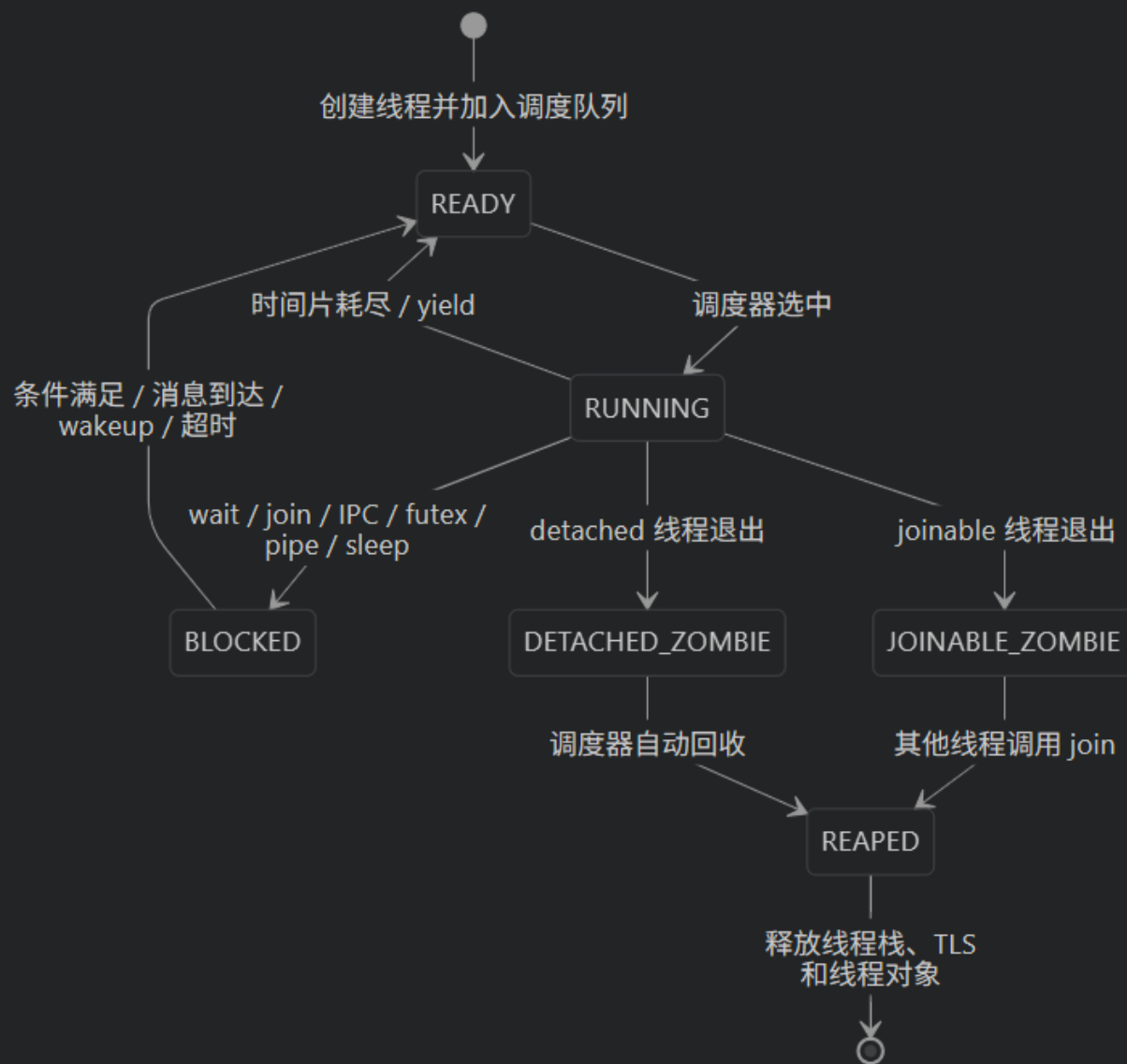
线程切换：保存旧线程寄存器和用户 RSP，选择新线程，恢复新线程寄存器和栈，进程变化时切换 CR3

fork：复制 VMA、文件对象引用、页表和 syscall frame

exec：替换当前用户地址空间，不改变 PID

exit：记录退出状态并释放进程资源

wait：取得子进程状态并回收 zombie



六、系统调用、IPC 与同步

系统调用

用户程序 (Ring 3)

read / write / fork / mmap

↓ SYSCALL

内核 (Ring 0)

检查参数 → 执行对应服务

↓ SYSRET

用户程序继续运行

系统调用是用户程序请求内核服务的统一入口。文件读写、进程创建和内存映射都要经过系统调用。用户地址先检查, 再通过 copyin / copyout 传递数据。

消息 IPC: 用于进程之间传递来源、类型和值

进程 A send(B, type, value) → 内核消息等待队列 → 进程 B receive(A)

线程同步: futex

36 个系统调用

基础 I/O: write、read、clear、get_ticks、sleep

文件目录: open、close、unlink、list、mkdir、stat、chdir、getcwd

进程: spawn、exec、fork、exit、wait、getpid、kill、ps

线程: thread_create、thread_join、thread_exit、gettid、thread_detach、thread_yield

IPC/同步: send、receive、futex_wait、futex_wake

虚拟内存: mmap、munmap

文件描述符: dup、dup2、pipe

七、持久文件存储：ATA LBA48 与 MyFS

文件访问路径

Ring 3 Application

↓ open/read/write/stat

File Descriptor

↓

Shared File Object

↓

FS Service + Block Cache

↓

MyFS

↓

ATA PIO LBA48

MyFS 磁盘格式

4 KB block

Superblock

Inode Bitmap

Data Bitmap

Inode Table

Data Blocks

inode大小: 72 B

directory entry大小: 64 B

文件块索引

11 个直接块+三级间接索引

路径支持: 绝对路径 + 相对路径

文件操作:

open/read/write/close/unlink/stat

目录操作: list/mkdir/chdir/getcwd

FD: fork/dup 后共享 file object 和 offset

Pipe: 环形缓冲、阻塞读写、EOF、readers/writers

ATA 驱动使用 LBA48 访问磁盘。MyFS 的超级块记录各区域位置，宿主机 mkfs 与 Guest 使用相同的 inode 和目录项格式。文件描述符、共享 file object 和 pipe 共同支持 Shell 的组合操作。

八、TTY 与 Shell 形成完整交互环境

键盘输入路径

8042 Keyboard IRQ



读取并解码 scancode



Keyboard Event Queue



TTY Input Service



Foreground Process read(0, ...)

显示路径

Application write(1, ...)



TTY / Console Cells



Framebuffer 或 VGA

framebuffer: 1280×720

支持滚屏、光标和 PSF 字体

crt0: 用户程序的启动入口, 负责调用 main

libc: 用户态基础函数库。

Shell: 内建命令与 ELF 程序

支持: argv 参数传递、A | B 管道

<、>、>> 重定向、& 后台执行

历史、Tab 补全

行编辑:

Ctrl+U: 清除当前行

Ctrl+W: 删除前一个单词

Ctrl+C: 终止前台进程

键盘中断只完成有界的扫描码读取和事件入队, 可能阻塞的读取由 TTY 服务在线程上下文中完成

九、测试

测试流程：

创建临时磁盘



重新构建内核和 MyFS



启动 QEMU



通过 QEMU monitor 发送真实按键



抓取 VGA 文本或 framebuffer 截图



检查输出、文件和资源状态



停止 QEMU并记录结果

#	测试名称	主要验证内容	通过标准
1	build.artifacts	MBR、Loader、Kernel、ELF、MyFS 制品	大小、格式和磁盘/物理边界正确
2	mkfs.index	MyFS 多级索引	5 MiB 文件正确使用一级和二级间接索引
3	mkfs.lba48	宿主机 mkfs 的 LBA48 计算	支持 FS 位于 128 GiB 以上
4	boot.quiet	普通启动路径	内核、存储和 Shell 正常，画面稳定
5	boot.dynamic-loader	Loader 动态读取内核	300 扇区内核通过 255+3 分块读取
6	lba48.boot	Guest ATA LBA48	Guest 能访问 128 GiB 以上的 MyFS
7	fs.service	MyFS 服务和持久化	文件操作成功，重启后数据存在，无资源泄漏
8	tty.shell	TTY 和 Shell 组合功能	管道、重定向、目录、清屏和滚屏正确
9	userland.core	crt0、libc 和用户程序	argv、libc、外部 ELF、ps、ls 等正常
10	input.stress	键盘快速输入和 Ctrl+C	20 条快速命令不漏字，前台进程可终止
11	sync.core	futex 和用户态同步	10 轮同步正确，无等待者和资源残留
12	vm.cow	mmap、COW 和页错误	父子写隔离、栈增长和回收正确
13	integration.smoke	多模块完整组合	启动、exec、IPC、同步、VM、fault 全部串通
14	showcase.core	六步现场演示	6 项 PASS，Shell 恢复，proof 文件跨重启
15	framebuffer.core	图形终端真正运行	PPM 尺寸、像素数量、背景和字形正确
16	shell.editor	Shell 行编辑器	插入、历史、Tab 补全和反向搜索正确
17	sync.stress	同步压力测试	100 轮× 4 个线程× 每线程 20,000 次更新



感谢聆听

李晨恺 2026.8.23